

News Bias Classification: Encoder vs Decoder Models

Arsenijs Golicins
Małgorzata Kwapniewska
Iulia Capralova

GitHub Repository

Abstract

This project investigates political bias detection in news articles using large language models (LLMs). We compared encoder-only (RoBERTa) and decoder-only (Gemma-2b) models across various tuning methods, including full fine-tuning, parameter-efficient LoRA, and prompt engineering with Gemma-2b-it. Results show that LoRA provides a high-accuracy, resource-efficient alternative to full fine-tuning, while prompt engineering offers a flexible, no-training option. We found that RoBERTa, as an encoder-only model, is more suitable for this task, highlighting the trade-offs between encoder and decoder models for bias detection and informing future applications in media analysis.

1 Introduction

In today’s media landscape, detecting bias in news articles is becoming increasingly important yet challenging [1] [2] [3]. News bias, whether explicit or subtle, has the power to shape public opinion, influence political discourse [4], and contribute to societal polarization [5]. A biased presentation of facts can amplify certain viewpoints while downplaying others, often leading to a distorted public understanding of events [6]. Social networking and media algorithms function on confirmation bias. They boost engagement by amplifying emotionally charged or divisive content. Studies show that such algorithms can create echo chambers, promoting extreme or hostile viewpoints toward out-groups, ultimately driving profitability but at the expense of balanced content exposure [7] [8]. For individuals, communities, and even governments, distinguishing between fact and bias in reporting is important for informed decision-making and responsible civic engagement.

Identifying bias in news articles is a challenging task, as it requires more than detecting overt opinions or emotionally charged language. Bias often emerges subtly through framing choices, topic selection, and implicit assumptions rather than direct statements [9]. Furthermore, bias classification is inherently subjective. The same content can be interpreted differently depending on cultural, social, or personal factors, making reliability challenging as annotators’ own perspectives influence the very notion of the definition of bias [10]. These makes traditional natural language processing (NLP) models struggle to capture bias nuances, as they require understanding the broader context, historical connotations, and cultural references that reveal underlying biases. Considering that they are trained on datasets with possible annotation biases, it leads to unfair or inconsistent detection outcomes [11]. Furthermore, scalability remains a concern. Bias-detection systems are limited to specific socioeconomic contexts, making them adaptable only within certain regions or timeframes. As political climates and social narratives shift, these models cannot be generalized, leading to fast outdated [12].

Given this, in our project we aim to develop a model capable of classifying political bias, focusing on comparing large encoder-only (RoBERTa) and decoder-only (Gemma) language models. The model should detect the ideological leaning of a text, classifying content as left, right, or center. By studying how each model type handles the intricacies of biased language, we hope to create a tool that not only detects bias, but also provides a reasoning for its classification, increasing transparency. Such a tool could help create a more balanced media landscape, allowing readers to better interpret

information amid rising concerns about bias and misinformation.

We want to compare encoder and decoder models’ performance because, although encoders are tailored to handle sequence classification (or token classification tasks) explicitly, recent progress in decoder-only transformers had shown that the same task can be accomplished with similar performance to encoders using those decoders [13]. When it comes to the prompt engineering, the difference becomes obvious - decoder-only models can be prompted to ”demonstrate” its reasoning [14]. The same behaviour cannot be manifested in encoder-only models, as they have a different objective function. Then, our point of inquiry was whether it is possible to obtain a better performance on Gemma (decoder-only model), than on RoBERTa (encoder-only model), given the task of sentence classification is native to encoder-only models. Should that be the case, decoder-only advantages, such as techniques derivative of Chain of Thought (CoT) could be utilised to improve the transparency of the models on the task.

2 Related Work

Text classification has long been a central task in NLP, used in sentiment analysis [15], spam detection [16], and document categorization [17]. Historically, LSTMs are one of the favored models for handling sequential data [18], since they can capture long-term dependencies in data [19]. Bai et al. explored LSTMs’ performance in text classification, particularly against CNNs and RNNs, for sentiment analysis [20]. The study found that LSTM outperformed the other two types of models (CNNs and RNNs), demonstrating its effectiveness in handling contextual semantics and mitigating the vanishing gradient issue inherent in RNNs [21].

However, LSTMs also come with certain limitations. Its sequential nature results in higher memory usage, longer training times, and reduced scalability [22]. This problem is investigated in a research paper written by Soyalp et al. [23], where the authors propose an improved text classification using the encoder-transformer model. It was concluded that transformer models have a higher rate of success compared with LSTMs and CNNs. Similarly, Baly et al. explored RoBERTa for predicting political bias in news articles [24]. Their study highlighted transformers’ ability to generalize across unseen media sources. They found that RoBERTa surpasses LSTMs in accuracy and efficiency but are also well-suited for complex, real-world applications.

Both Soyalp et al. and Baly et al. focused on encoder-only transformers, as training an encoder-only model has shown comparable results to an encoder-decoder setup. A research paper written by Liu et al. demonstrated that encoder-decoder models are resource-intensive and often introduce unnecessary complexity for classification tasks, which do not require sequence generation [25]. This has led to a shift toward encoder-only models, streamlining the architecture by focusing mostly on encoding for classification tasks [26].

More recently, however, decoder-only models like GPT-3 have emerged as viable alternatives in text classification and sentiment analysis. Although primarily designed for generative tasks [27], decoder-only models can be adapted for classification through prompt-based learning. It was demonstrated in a study by Pawar et al., where the performance of GPT-3 was compared to BERT on a sentiment classification task [28]. The results demonstrated that GPT-3 outperformed the BERT-base model in classifying text polarity.

Although some studies, such as those using RoBERTa, have explored news bias detection, this remains a relatively understudied field, with existing research limited in scope. Due to the rapidly evolving nature of news media and shifting political landscapes, most studies rely on small or narrow datasets that quickly become outdated.

News bias detection also demands models capable of understanding complex ideological nuances, contextual cues, and subtle framing shifts—challenges that decoder-only models like Gemma are well-suited to address. Decoder models leverage prompt-based learning, allowing for a flexible interpretation that adapts to nuanced contexts, making them potentially powerful tools for detecting evolving biases in media.

Yet, there remains a clear research gap in directly comparing encoder-only and decoder-only models in this domain. Most existing work focuses on encoder models, with limited exploration of how decoder models perform in capturing media bias. Our study aims to address this gap by providing a head-to-head comparison of RoBERTa (encoder-only) and Gemma (decoder-only) on the task of detecting political bias in news. By examining each architecture’s strengths and limitations, we aim to deepen our understanding of how encoder and decoder models handle the unique complexities of media bias detection.

3 Methods

We selected two models for our comparative study: RoBERTa_{BASE}, an encoder-only model, and Gemma-2b, a decoder-only model. RoBERTa is well-suited to tasks focused on input comprehension, while Gemma-2b, as a generative model, allows for output introspection and reasoning. Our goal was to examine whether differences in performance emerge between these architectures for bias direction classification.

For each of the chosen models, the pre-trained general model was used as a baseline, with a linear classification layer added to map model outputs to three labels: 0 (right-leaning), 1 (center), and 2 (left-leaning). Initially, performance was expected to be close to random due to the lack of task-specific fine-tuning. Subsequently, we fine-tuned each model using a parameter-efficient fine-tuning technique, LoRA, which allowed us to adapt the models while preserving their original parameters. Further, fine-tuning was conducted until each model approached overfitting on the validation set, ensuring optimal performance for comparative analysis. Lastly, a prompt engineering technique based on reasoning induction, called Clues and Reasoning Prompting (CARP), was used on the encoder-only model. The following section describes each of the models, as well as the training techniques.

3.1 RoBERTa

3.1.1 Model description

RoBERTa, developed by Meta, is an encoder-only transformer model optimized for robust language understanding and classification tasks. It was introduced as an enhancement of BERT, aimed at achieving more reliable representations through modifications to BERT’s original training [29]. With approximately 120 million parameters in its base version, RoBERTa increases model efficiency and performance on various NLP tasks.

Like BERT, RoBERTa is trained using Masked Language Modelling objective function. That is, parts of the inputs are randomly masked, and the objective is to predict the words that were masked. The model is then forced to consider bidirectional dependencies of the context. The loss function for training encoder-only models is given below.

$$\text{Loss} = - \sum_{i \in M} \log P(x_i | X_{\text{masked}}), \tag{1}$$

where M is the set of masked positions. For the purpose of our task, we have to classify the entire sequence, that is, predict its value (0, 1, 2), not just one word. In encoders, usually the very first output token is serving as the token for classifying the whole sequence. That means that fine-tuning of RoBERTa on our task will be a cross-entropy loss function (see Section 4.4).

To improve the performance of the pre-trained model on downstream tasks, RoBERTa was designed with architecture similar to BERT, described in the section below, but trained on a significantly larger dataset. It included BookCorpus, English Wikipedia, CC-News, OpenWebText, and Stories, covering over 160GB of text data [30]. This extensive training allows RoBERTa to develop an understanding of general language patterns, syntax, and semantics. Furthermore, the training epochs were enlarged, and some of the objectives from BERT were removed, with the most significant being

the Next Sentence Prediction (NSP) task. The research found that removing the function does not hinder the performance of the pre-trained model, but its removal significantly improves the ease of fine-tuning for downstream tasks [29]. Additionally, RoBERTa uses larger batch sizes and increased sequence lengths during pretraining, resulting in improved contextual understanding and resilience to complex linguistic nuances[29].

These optimizations make RoBERTa particularly well-suited for classification tasks, as its focus on deep bidirectional encoding allows it to effectively capture underlying biases in news articles by processing full sentence structures and contextual dependencies. Furthermore, optimisation from the base BERT model make it suitable for robust fine-tuning and more applicable to downstream tasks, such as news classification, as showcased by numerous studies utilising RoBERTa for various text-classification tasks [31] [32].

3.1.2 Model architecture

RoBERTa_{BASE} architecture is identical to Google’s BERT_{BASE}’s, which is a bidirectional encoder. BERT was inspired by the successes and limitations of two different models: ELMo and OpenAI’s GPT. ELMo was the first large bidirectional model, capable of capturing the entire sequence at once, allowing for detailed capturing of long-term dependencies between the words [33]. Despite this innovation, ELMo lacked attention mechanisms, which limited its ability to focus on specific words or phrases that could shift the meaning of a given word within its context. GPT, on the other hand, utilised transformer-based multi-attention head, which allowed for better context-capturing in NLP tasks. GPT, however, is a one-directional model, making it heavy dependent on context size window and allowing for one-direction of context computation[31].

BERT’s (and, in extension, RoBERTa’s) architecture was designed to make use of both major advantages of the described above models: transformer attention head allows for accurate capturing of a given word’s influence over the meaning, while the bidirectional approach enables between-layer context transfer[31].

Consequently, RoBERTa is constructed from 12 stacked transformer blocks, each with its own attention block, consisting of $A = 12$ attention heads. [29]. Due to the increased complexity coming from presence of attention heads, the depth of neural network had to be increased in comparison to BERT to $H = 768$ layers [31].

3.2 Gemma

3.2.1 Model description

For our comparison, we selected Gemma as the decoder-based model. Developed by Google, Gemma 2 is available in three sizes: 2, 9 and 27 billion parameters [34]. Due to computational constraints, we used the 2-billion parameter version, referred to as Gemma-2b. This model functions as a text-to-text large language model (LLM) and is available in both pre-trained and instruction-tuned versions. We employed the pre-trained version for most of our methods but used the instruction-tuned variant (indicated by the `-it` flag) for our prompt engineering approach, as the default model does not follow instructions to the level allowing for CARP application. Gemma-2b-it includes additional training on structured, conversational, and instructional texts [35].

Gemma was trained on a dataset of English text from web sources, totaling 2 trillion tokens, to develop a comprehensive understanding of language patterns [36]. The original training dataset includes various sources: web documents, code, and mathematics, which helps the model develop skills in logical reasoning and symbolic representation. This extensive pre-training enables Gemma-2b to be a general-purpose language model, adaptable across a variety of tasks. Text was tokenized with a customized SentencePiece tokenizer[37]. The instruction-tuned version of Gemma is further optimised with two fine-tuning techniques: supervised fine-tuning (SFT) and reinforcement learning from human feedback (RLHF). These additional training steps improve its ability to respond accurately to prompts[38]. Furthermore, the smaller (2b and 9b) models utilised knowledge destination

- training with supervision from the large (27b) model. Such technique allowed for efficient training with smaller number of parameters.

As a decoder-only model, Gemma produces one token at a time in a continuous sequence generation task, based on the previous context computed unidirectionally[38]. This approach follows the Causal Language Modelling framework (CLM)[39], where the objective is maximising the likelihood of the next token given the previous token sequence. Hence, the loss function is:

$$\text{Loss} = - \sum_{i=1}^n \log P(x_i | x_1, \dots, x_{i-1}) \quad (2)$$

For our purposes, the loss for fine-tuning Gemma can be expressed as cross-entropy (see Section 4.4).

3.2.2 Model architecture

Gemma-2b is a light-scale version of the Gemini model, with architecture following the well-established transformer design[40]. Parameters are $L = 18$, $H = 32768$, $A = 8$. The vocabulary size of Gemma-2b is 256128 and the embedding size is 2048. Out of 2.5 billion parameters, 20% are embedding parameters and the rest 80% are non-embedding parameters [38].

Gemma introduced several improvements over the original Transformer decoder architecture from Vaswani et al. (2017). One key advancement is the use of Multi-Query Attention (MQA) instead of traditional Multi-Head Attention. MQA simplifies the attention mechanism by allowing each attention head to share a single set of keys and values while maintaining unique query representations in each attention head. This reduces memory and computational overhead, making the model more efficient[34].

In addition to this, Gemma replaces traditional positional embeddings with Rotary Positional Embeddings (RoPE), which are applied at each layer and shared between inputs and outputs. Unlike fixed positional encodings, RoPE allows for more flexible and scalable encoding of positional information, improving the model’s performance on longer sequences by encoding positions in a continuous, rotational space [41].

Further, in its feed-forward layers, Gemma uses Gated Linear Units with GeGLU activation functions [42]. This gated mechanism introduces an additional gating factor, allowing the model to modulate information flow more effectively than standard ReLU activations. This improves the model’s expressiveness, improving on traditional methods of introducing non-linearity into models.

Finally, instead of using Layer Normalization, which is typical in many Transformer architectures, Gemma implements RMSNorm [43] for normalization within Transformer layers. RMSNorm normalizes input vectors over their root mean square values, which has been shown to improve training stability and convergence speed, particularly in large models.

3.3 LoRA

As model sizes increase, full fine-tuning becomes increasingly impractical due to the high computational and memory demands. However, recent work has shown that comparable performance can be achieved with parameter-efficient fine-tuning methods such as Low-Rank Adaptation (LoRA). Hu et al (2021) showed that LoRA can reduce the number of trainable parameters by 10,000 times and the GPU memory requirement by three times [44]. This efficiency improvement was one of the motivations for integrating LoRA-based training into our project.

Another reason for choosing LoRA over other PEFT techniques like prefix-tuning was its compatibility with different model architectures, a necessary requirement given the comparative nature of the project. LoRA modifies weight matrices directly, which allows for fine-tuning without the need for changing either input structures or modifying the token lengths. Its usage was therefore an optimal choice for a study with multiple models. Furthermore, operations on weight matrices are more suitable for classification tasks, while prefix-based methods are typically applicable to text generation.

LoRA’s focus on parameter decomposition further aligns with our goal to classify bias direction in news articles, as it allows us to keep the training lightweight without introducing excessive parameters. Specifically, LoRA decomposes parameter matrices into smaller matrices, reducing computational cost[44]. For example, a 100x100 parameter matrix (10,000 parameters) can be represented as the product of a 100x2 and a 2x100 matrix, totaling only 400 parameters. Given that classification tasks require efficient feature representation at the token level without expanding input sequences, LoRA’s matrix-based approach makes it particularly efficient for encoder models, where the first token often aggregates sequence information.

Moreover, the research shows that low dimension reparameterisation can be as effective as full fine-tuning [45], as LoRA’s decomposition enables direct training on the downstream classification task without adjusting the input structure. As such, we expect the PEFT decomposition to bring similar performance to full fine-tuning method, while using much smaller computational resources. In our case, for both RoBERTa and Gemma-2b, LoRA application shrunk the size of trainable parameters to only 0.9% of the original parameter set size.

3.4 CARP

Prompt engineering approaches have enhanced the classification capabilities of decoder-only models. Research indicates that prompting techniques requiring reasoning behind decisions significantly reduce the model’s tendency to hallucinate [46]. For instance, Clue And Reasoning Prompting (CARP) uses a progressive reasoning strategy, first prompting the model to identify superficial clues, then reasoning with these clues for final classification [47]. CARP has achieved state-of-the-art results, demonstrating how advanced prompt engineering can leverage decoder-only models’ generative abilities for classification tasks.

This is why our methodology also incorporates the CARP approach. CARP is a chain-of-thought prompt engineering method, where the model is guided to improve its performance by following a clear sequence of reasoning steps. This structured approach aids the model in handling complex relationships in text classification by defining a systematic process for extracting reasoning. The CARP method involves three stages:

1. Find superficial clues (e.g., keywords, tones, semantic relations, references, etc)
2. Based on the clues, the LLM is asked to reason potential bias based on keywords that it identified in the text. Since the process is instigated by self-identified clues, there is a smaller chance of model hallucination. Moreover, flaws in the openly presented reasoning process are easier to spot.
3. Produce the final decision based on the outcome of the reasoning process.

We expanded CARP into three variants for our task. Standard CARP prompts the model to determine the overall political bias as left, right, or center. For comparison, we introduced a 2-label CARP condition, removing "center" samples and asking the model to classify only "left" or "right" bias, based on the assumption that "center" can be ambiguous. Finally, the "logical deduction" CARP condition explicitly instructs the model to base its bias inference on identified clues. Examples of each CARP prompt variant are shown in the text boxes below.

Standard CARP

This is a political bias classifier for news headlines.
Please provide the response in the following JSON format: {
"Clues": "<List the CLUES here (i.e., keywords, tones, references, contextual information)>",
"Reasoning": "<Provide the REASONING process here in 130 words based on the clues and input>",
"Political Bias": "<Determine the overall POLITICAL BIAS as Left, Right, or Center. You MUST write only one word in this section either right, left or center.>"
} INPUT:

2-label CARP

This is a political bias classifier for news headlines.

Please provide the response in the following JSON format: {

"Clues": "<List the **CLUES** here (i.e., keywords, tones, references, contextual information)>",

"Reasoning": "<Provide the **REASONING** process here in 130 words based on the clues and input>",

"Political Bias": "<Determine the overall **POLITICAL BIAS** as either **Left** or **Right**. You **MUST** write only one word in this section either **Left** or **Right**.>"

} INPUT:

Logical CARP

This is a political bias classification task for news headlines. Your goal is to **analyze** the headline to identify relevant information and **logically deduce** the political bias.

Please provide the response in the following JSON format: {

"Clues": "<Identify specific **CLUES** (such as keywords, tones, references, or contextual elements) from the headline that can be used to determine the political bias.>",

"Reasoning": "<Explain the **REASONING** behind the bias determination, detailing **how the clues support your conclusion**. Limit the reasoning to 130 words.>",

"Political Bias": "<Logically infer overall **POLITICAL BIAS** as Left, Right, or Center. You **MUST** write only one word in this section either right, left or center.>"

} INPUT:

4 Experiments and Results

4.1 Data Source and Inspection

In this project we used *AllSides* dataset. It was collected from *allslides.com*, which is the aggregator of news coverage that has experts tagging ideological affiliation of the viewpoints [48].

Dataset contains 21,747 U.S. news samples, where each sample shares a unique title (news topic) with two others. That is, there are 7,249 unique titles, with 3 different coverage of each. Each sample contains the title, tags (which are shared for a given title), heading, source, text, and bias rating. The bias rating, that is, political categories that news headings are classified into are left, right, or center. For bias rating, there are 10,273 (47.2%) samples tagged as left (encoded as 0), 7,222 (33.2%) tagged as right (encoded as 2), and 4,252 (19.6%) as center (encoded as 1).

4.2 Data Preprocessing

Data preprocessing begins with data loading, where the dataset is eagerly loaded from a specified directory. After loading, following preprocessing steps were taken:

1. Only essential columns are selected, specifically heading, text, and bias rating. 'Title' and 'Tags' were omitted since they were shared between samples of different political coverage, which served for the classification of the news topic, but could worsen model performance. Additionally, the 'Source' was not used since it might introduce unnecessary data leakage, as certain sources can be directly linked to bias without looking at the text content. These columns are retained as they provide the core information needed for classification, and later, the heading and text columns are combined into a single column, heading text, to facilitate unified processing.

2. Missing values are addressed by removing rows with incomplete data, as substituting or reconstructing text entries is infeasible. In text datasets, each data point contains context that cannot be replicated or inferred. Next, duplicates are removed from the dataset, keeping only the first occurrence of each unique text.

3. Several steps were taken to normalize text in our dataset. First, any ambiguous characters in the text are corrected, and non-ASCII characters are replaced with their ASCII equivalents. Following this, all text is converted to lowercase, maintaining consistency across the dataset.
4. Extra whitespace within and around sentences is removed to create a cleaner, more uniform text structure.
5. The label column, is then converted to numerical labels, mapping left to 0, center to 1, and right to 2, creating a format compatible with model training.
6. To prepare the data for model input, tokenization is performed using specific tokenizers for each model to ensure compatibility and effective text processing. For the Gemma 2b model, we use the GemmaTokenizerFast, which is optimized for Gemma’s vocabulary and model structure. For RoBERTa, the AutoTokenizer from is applied, ensuring the text aligns with RoBERTa’s requirements for embedding and contextual representation. These tokenizers transform the cleaned and normalized text data into sequences of tokens that both models can interpret, facilitating consistent input for training and classification.
7. We split our dataset into training and test sets using an 80/20 ratio, resulting in 17,397 samples for training and 4,349 samples for testing. This split maintains a sufficient sample size for training and testing.

4.3 Evaluation Metric

Accuracy was selected as an evaluation metric to assess the performance of our models in detecting political bias in news articles. Accuracy is a widely used metric in classification tasks, representing the proportion of correctly classified instances out of the total instances[49]. It is calculated as follows:

$$\text{Accuracy} = \frac{\text{Correct predictions}}{\text{Total predictions}} \quad (3)$$

In our project, accuracy measures the model’s ability to correctly classify news articles into bias categories (left, center, right). Given the task, accuracy is an appropriate metric as it reflects the model’s performance in assigning correct categories without penalizing for bias intensity misclassification.

4.4 Loss metric

In training RoBERTa and Gemma cross-entropy loss is used as the loss function. It penalizes the model for incorrect predictions and optimizes it to improve prediction probabilities for the correct class labels. It is implemented as a combination of two steps: a softmax function to convert logits into class probabilities, followed by the Negative Log-Likelihood Loss (NLLLoss) to measure the error [50]. For each input, the softmax layer transforms the raw logits z_i (where i indexes the classes) into probabilities $\hat{y}_i$ for each class C :

$$\hat{y}_i = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}} \quad (4)$$

Once the probabilities are computed, NLLLoss then calculates the loss for the true class label. Assuming y is the true label, NLLLoss can be defined as:

$$L = -\log(\hat{y}_y), \quad (5)$$

where $\hat{y}_y$ is the predicted probability of the correct class y . The negative log probability penalizes the model when $\hat{y}_y$ is low, with the loss decreasing as the predicted probability for the true label

increases. For multi-class classification, cross-entropy loss generalizes this by summing across all classes C :

$$L = - \sum_{c=1}^C y_c \cdot \log(\hat{y}_c), \quad (6)$$

where y_c represents the one-hot encoded vector for the true class label, which has a value of 1 for the correct class and 0 for all other classes. Log probability of the correct class influences the loss, pushing the model to improve its confidence in the correct label over time.

4.5 Training RoBERTa

This section describes the hyperparameter tuning and training procedure of each type of model. Results are shown in Table 3 and are further discussed in Section 4.7.

4.5.1 Baseline - Pre-trained

As our baseline, we used a pre-trained RoBERTa model directly on our test dataset for news bias detection, with no further training or weight updates on our data. By applying this model to our task without additional fine-tuning, we aim to evaluate how well RoBERTa’s pre-existing language understanding can generalize to the specific task of news bias detection. This way we can understand RoBERTa’s ability to detect bias in news content based purely on its broad, pre-trained knowledge base, serving as a benchmark for comparison with further fine-tuned models.

4.5.2 Fine-tuning

Fine-tuning is used to adapt a pre-trained model like RoBERTa to a specific task, such as news bias detection because it enables the model to go beyond generic language understanding and specialize in identifying patterns unique to the new data. While RoBERTa is trained on vast amounts of general language data (see Section 3.1.1), it is not specifically designed to detect subtle ideological cues or patterns of bias inherent in news text. By fine-tuning all 125,830,662 parameters, we allow the model to adjust its representations fully, aligning them with the specific linguistic structures relevant to our task.

In order to improve the predictive performance of the model, we performed hyperparameter tuning through grid search. We experimented with a learning rate that controls the step size during weight updates and significantly impacts the model’s convergence rate and stability during training. Choosing a learning rate that is too high can lead to divergence, while a low learning rate can cause prolonged training times[51]. Range of values for learning rate, shown in Table 1b, aiming to capture an optimal balance between training speed and stability. The batch size, which determines the number of samples processed before updating model weights, influences both training stability and computational load. Smaller batch sizes can yield more frequent updates but may result in less stable gradients, while larger batch sizes reduce update frequency but improve gradient stability. By evaluating batch sizes of 8 and 16, we aimed to find an optimal batch size that maintains computational efficiency while ensuring effective learning. The most optimal values found were: LR = 0.00002 and batch size = 8.

Parameter	Search space
LR	0.00005, 0.00003, 0.00002
Batch size	16, 8

(a)

Parameter	Search space
LR	0.00005, 0.00003, 0.00002
Batch size	16, 8
Alpha	16, 32
R	8, 16
Dropout	0.1, 0.2

(b)

Table 1: (a) Grid search space for the fine-tuning and (b) parameter efficient fine-tuning with LoRA for RoBERTa

4.5.3 LoRA

Fully fine-tuning the RoBERTa model is computationally expensive, as explained in Section 3.3, therefore, we applied parameter-efficient fine-tuning. It allows us to update only 1,182,723 parameters, which represents just 0.9% of all trainable parameters in the RoBERTa model. To maximize the performance of RoBERTa with LoRA, it is important to tune not only the model’s core hyperparameters but also the specific parameters introduced by LoRA.

In training RoBERTa with LoRA, we reused the same strategy for tuning the learning rate and batch size from the previous RoBERTa fine-tuning grid search (see Section 4.5.2), as the core model remains the same. First, we tuned alpha, the scaling factor that controls the magnitude of the low-rank updates applied by LoRA. It influences the low-rank updates in the overall model adjustment process. A lower alpha can result in underfitting, where the updates have minimal impact on the model’s ability to capture new patterns, while a higher alpha can lead to overfitting or instability by making the updates too prominent. We tested values of 16 and 32 to balance sufficient adaptation with model stability, aiming to improve performance without causing excessive adjustments to the model’s weights.

Next, we tuned the rank R , which determines the dimensionality of the low-rank decomposition in LoRA. The rank controls the flexibility of the updates that LoRA applies to the model. A higher rank increases the model’s ability to capture complex patterns by allowing more detailed updates, but it also requires more memory and computation. On the other hand, a lower rank reduces computational cost but may limit the flexibility of the model in adapting to new data. By testing values of 8 and 16, we aimed to find a balance where LoRA could apply updates that were complex enough for our dataset.

Finally, we applied dropout with values of 0.1 and 0.2 as a regularization technique to prevent overfitting. Dropout helps the model generalize by randomly disabling a fraction of neurons during training, which reduces reliance on specific parts of the network. This regularization step is important in fine-tuning tasks to avoid overfitting to the training data.

The most optimal values found were: LR = 0.00002, Batch size = 8, Alpha = 32, R = 16, Dropout = 0.1.

4.6 Training Gemma

This section describes the hyperparameter tuning and training procedure of each type of model. Results are shown in Table 3 and are further discussed in Section 4.7.

4.6.1 Baseline - Pre-trained

As part of our baseline evaluation, we tested the performance of a pre-trained Gemma-2b model directly on our test dataset. We followed the same testing procedure as described in Section 4.5.1. By applying Gemma-2b to our news bias detection task without additional fine-tuning, we aim to assess how well its broad, pre-trained knowledge generalizes to bias detection.

4.6.2 Fine-tuned

To adapt the Gemma-2b model with approximately 2.5 billion parameters to our news bias detection task, we fine-tuned only 50% of its parameters. Due to computational constraints, we were unable to train all parameters, as the high memory and GPU demands of Gemma-2b exceeded the resources available on Habrok. Training one epoch took approximately one hour, which limited us to performing full hyperparameter tuning through grid search. Instead, we adopted a trial-and-error approach, which allowed us to find suitable values for hyperparameters. Similar as in Section 4.5.2 we experimented with batch size and learning rate, since they influence model convergence, training stability, and overall performance. We tried three value configurations which are shown in Table 2a.

(a) Configuration settings for learning rate and batch size to fine-tune Gemma-2b			
Parameter	Config 1	Config 2	Config 3
LR	0.00005	0.00003	0.00002
Batch size	8	8	16

(b) Gemma-2b LoRA Configurations			
Parameter	Config 1	Config 2	Config 3
Learning Rate	0.000005	0.000003	0.00002
Batch Size	8	8	16
LoRA Rank (r)	16	8	16
Alpha	32	16	32
Dropout Rate	0.1	0.2	0.15

Table 2: Configurations for hyperparameter tuning for (a) fine-tuning Gemma and (b) Gemma with LoRA

The chosen values for learning rate and batch size span a range that is effective for fine-tuning large models. Configuration 1 uses a learning rate of 0.00005 with a batch size of 8, which we expected to provide faster convergence while keeping the batch size manageable to maintain gradient stability. Configuration 2 lowers the learning rate to 0.00003 while keeping the batch size at 8, aiming to reduce the risk of overshooting optimal weights by taking more gradual steps in training. Configuration 3 uses a smaller learning rate of 0.00002 and a larger batch size of 16, balancing slower updates with greater stability from the increased batch size. After testing each setup, we found that Configuration 1 demonstrated the best performance, achieving a balance between fast convergence and training stability.

4.6.3 LoRA

To adapt Gemma-2b to our news bias detection task in a computationally efficient manner, we used Low-Rank Adaptation (LoRA). It allowed to fine-tune only 0.001 billion parameters, representing approximately 0.93% of the total parameters in Gemma-2b. This limited number of trainable parameters allows us to retain most of Gemma-2b’s pre-trained language understanding while adapting it to detect bias in news content. Due to computational constraints described in Section 4.6.2, we opted for a trial-and-error approach instead of grid search, testing only three configurations of key hyperparameters: batch size, learning rate (LR), LoRA rank (r), alpha, and dropout rate. They are shown in Table 2b.

In configuration 1 we set a batch size of 8 and a learning rate of 0.000005, which allows for fine-grained updates to the model. With a LoRA rank of 16 and alpha set to 32, this configuration provides the model with more flexibility to capture task-specific patterns, while the 0.1 dropout rate helps prevent overfitting. This setup aimed to strike a balance between efficient learning and regularization. Configuration 2 has increased the batch size to 16 while reducing the learning rate to 0.000003 and halving both the LoRA rank to 8 and the alpha to 16. This setup was designed to be more conservative, reducing the extent of the model’s updates to improve stability, especially with a higher dropout rate of 0.2. We expected this configuration to offer better generalization at the cost of slower adaptability. Finally, third configuration combines a batch size of 8 with a lower learning rate of 0.000002, aiming for gradual, stable updates. We kept the LoRA rank at 16 and alpha at 32, similar to Configuration 1, while setting a moderate dropout rate of 0.15 to balance regularization with learning flexibility. It was aimed to improve stability without heavily sacrificing adaptability.

Each configuration was chosen to test how variations in these parameters impact model performance while staying within computational constraints. After testing, we found that Config 1 achieved the best performance, providing an optimal balance between learning rate, flexibility, and stability for our task.

4.6.4 Prompt Engineering

For prompt engineering, we used the Gemma-2b-it model instead of the original Gemma-2b, as it is specifically optimized for prompt-based tasks see Section 3.2.1. In our project, we did not modify any model weights or perform additional training. We leveraged the pre-existing capabilities of Gemma-2b-it to respond to various prompt formats. This approach allowed us to test how well prompt engineering works without changing the model’s existing knowledge.

As described in Section 3.4, we experimented with three different prompt variations to assess how each affected the model’s bias detection performance. The output for each data point was saved in json file, from which the label was extracted during accuracy calculation. Elaborate discussion of the results is included in Section 4.7

4.7 Results

The results of the run experiments provide insights into the effectiveness of the employed training techniques on the downstream task of bias classification. The juxtaposition of chosen models allows for comparison between decoder-only and encoder-only architectures, while the performance of each model across these methods highlights techniques’ respective strengths and limitations in adapting to our classification task. The models are compared based on accuracy as a performance measure. Table 3 summarizes the results, which will be further discussed in this section.

Method	Gemma-2b	RoBERTa
Pretrained (Baseline)	38.0%	32.5%
Fine-tuned	56.9%	62.7%
Lora	64.1%	61.7%
Prompt Engineering (standard CARP)	40.4%	-
Prompt Engineering (standard CARP) 2 labels	45.1%	-
Prompt Engineering (logical deduction CARP)	34.7%	-

Table 3: Accuracy results for different methods.

The results show that the baseline performance for both models was close to a random chance (0.33% for the three available labels), with 38% for Gemma and 32.5% for RoBERTa. This is expected, as the model was not trained for this classification task and was unaware of the correct labelling distribution.

Applying standard fine-tuning improved both models’ performance, although the results varied significantly between the two. Fine-tuning RoBERTa yielded a higher accuracy of 62.7%, making it more effective on our relatively small dataset. This is likely because RoBERTa, with fewer parameters than Gemma-2b, could be trained more efficiently without overfitting. In contrast, fine-tuning Gemma-2b, a much larger model, proved inefficient due to the dataset’s limited size relative to its parameter count. As a result, Gemma-2b quickly overfitted to the training data, with its test accuracy peaking at just 56.9%.

After the spotted inefficiencies in full fine-tuning, we will discuss the results from LoRA, a technique aimed at efficiency. LoRA significantly enhanced the performance of Gemma-2b, pushing its accuracy to 64.1%—the highest achieved in this study. This improvement highlights LoRA’s ability to

make larger models more adaptable to specific tasks without requiring extensive data. For RoBERTa, LoRA also yielded a strong result, achieving an accuracy close to its fine-tuned counterpart at 61.7%.

The training and validation loss curves for both models with and without LoRA further reinforce these findings (Fig 2 and 1). In standard fine-tuning, both models show consistent loss reduction across epochs, though with occasional signs of overfitting. With LoRA, however, the training and validation loss trends become smoother and more stable, indicating improved generalization. Furthermore, LoRA allowed for the longest epoch iteration for RoBERTa, where 7 training epochs were possible before the early-stopping mechanisms detected overfitting. This pattern is also visible in Gemma-2b, where LoRA’s stability pairs with the strongest accuracy, suggesting that the method helps the model adapt effectively without overfitting.

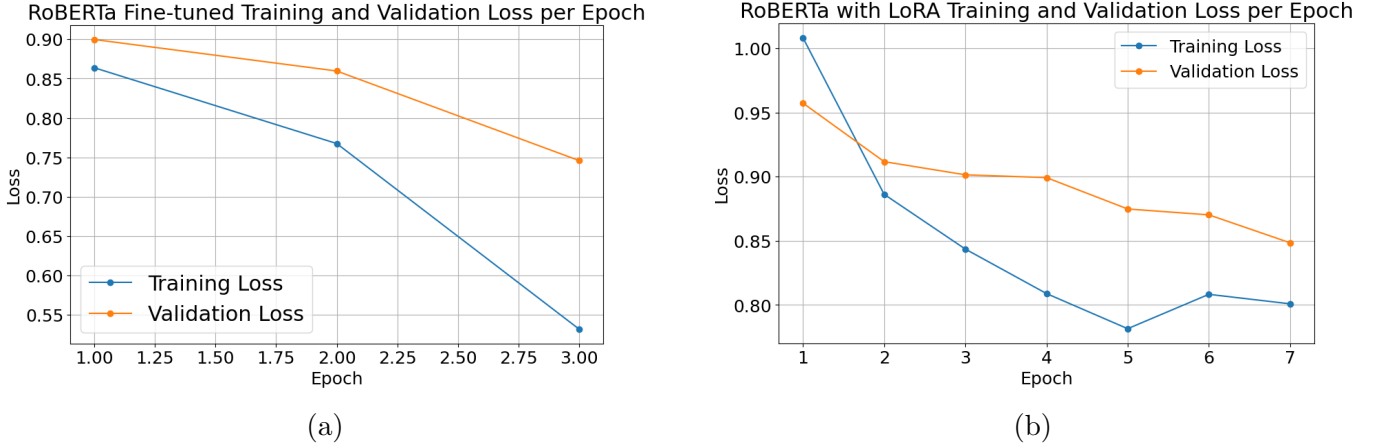


Figure 1: Training and test loss of the (a) fine-tuned RoBERTa and (b) RoBERTa with LoRA

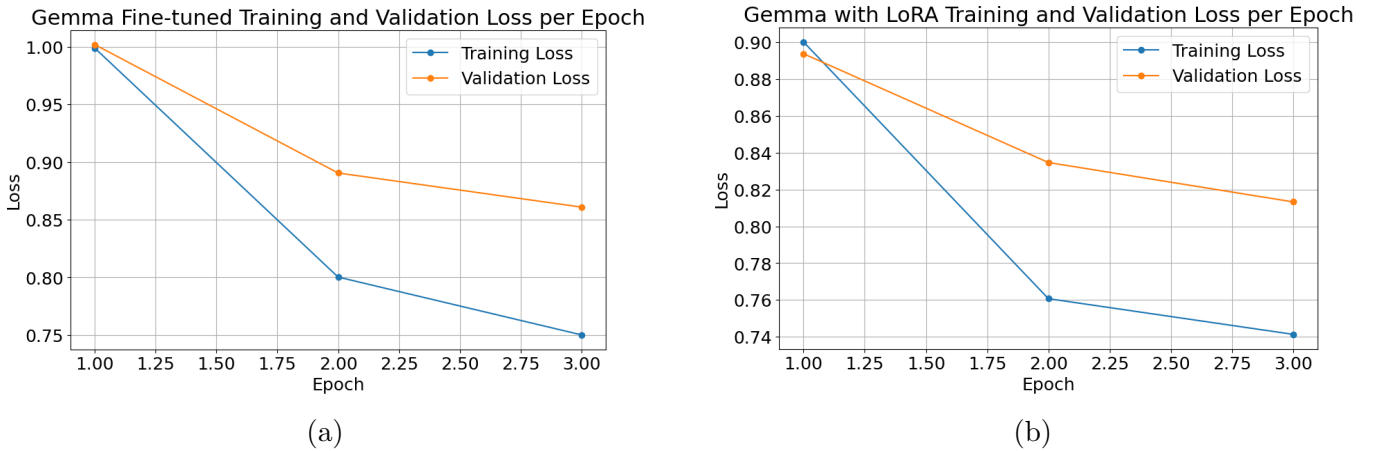


Figure 2: Training and test loss of the (a) fine-tuned Gemma and (b) Gemma with LoRA

5 Analysis and Discussion

5.1 Result Analysis

In their pretrained state, both Gemma-2b and RoBERTa demonstrate limited capability on our classification task, with accuracy scores of 38.0% and 32.5%, respectively. This reflects that, without any task-specific tuning, neither model can capture the nuances required for accurate classification. The low baseline accuracies showcase the necessity of applying fine-tuning techniques to achieve any reliability in performance.

Relatively high performance of RoBERTa aligns with expectations, as the bidirectional nature of these encoder-only models allows them to consider each word in the context of its surroundings,

which helps with a comprehensive approach to classification. In contrast, models with decoder-only architectures lack this bidirectional capability, as their masking mechanism prevents them from building contextual understanding in both directions simultaneously. Consequently, RoBERTa’s architecture, therefore, seems especially suited for tasks that require a holistic understanding of text.

Our experiment also highlighted the effectiveness of LoRA (Low-Rank Adaptation) as a fine-tuning approach, particularly for Gemma-2b. LoRA allowed Gemma-2b to reach an accuracy of 64.1%, outperforming standard fine-tuning and showing that LoRA can prevent overfitting while still adapting the model to the task. This suggests LoRA is not only efficient but also effective, especially in cases where there is limited data or computational resources. For RoBERTa, LoRA yielded results close to those achieved with full fine-tuning, but with a lower computational cost, reinforcing its value as an efficient fine-tuning strategy.

On the other hand, prompt engineering, using the CARP method, did not perform as well as fine-tuning approaches (Fig. 3). This result suggests that CARP might not be the best choice for bias classification, as it seems to lack the nuanced control needed to handle the often ambiguous language used in political texts. A closer inspection of produced justifications for incorrect labels shows that the reasoning clearly overfitted by choosing as clues issues mentioned most often by each political party. For example, an article informing about Donald Trump’s rally, where the issue of immigration was mentioned, was always classified as right-leaning, despite the reporting purpose of the article, and ignoring the sentiment with which the issue was mentioned. The underwhelming performance of CARP shows that simply prompting the model without any in-depth tuning may not be enough to guide it in detecting political biases effectively.

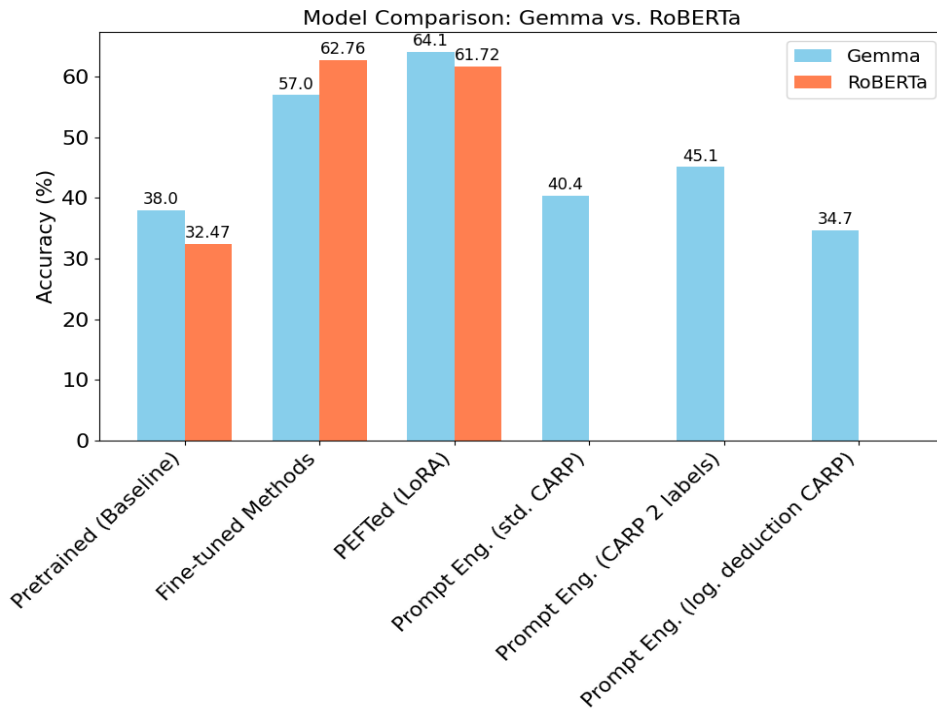


Figure 3: Overview of the accuracy of Gemma and RoBERTa models.

5.2 Further considerations

It remains an issue for discussion how the model maps certain features to the given political bias. The task of bias classification and its leaning (left or right), might in fact be reduced to loaded language and applying certain predicates (positive or negative) to certain concepts. For example, phrases like "legal abortion", "illegal abortion", "efficient corporations", "greedy corporations" carry implicit judgments that hint at political leanings. Under this assumption, "center" bias would map to neutral descriptions, not involving any assertions. This raises the question of whether truly non-assertive news coverage exists in the world of politics, where all decisions directly impact people in significant ways.

We have mentioned that the 2-label CARP condition was introduced due to the possible ambiguous mapping of "center" bias. One can easily see how defining "center" as an absence of bias can be problematic and somewhat arbitrary. If text is politically "non-biased", it may simply reflect a position aligned with the current status quo, which in itself can carry implicit biases. A reader from a different societal context may interpret the neutral stance differently. For example, if in a society A, where abortion is banned, an article written about it in an objective ("center"), euphemistic way, not expressing positive or negative sentiment, a reader from a society B, where abortion is allowed, might still see this coverage as, for example, expressing right political bias.

This relativity in interpreting "center" bias underlines a core challenge in political bias classification, and was reflected in the models' struggles with the 'center' label. The same language may be perceived differently based on the cultural or ideological perspectives of the reader. This observation suggests that bias classification models may need more sophisticated mechanisms to account for cross-cultural and contextual variations in interpreting political stances, especially for topics that are contentious and highly context-dependent.

6 Conclusion

6.1 Main findings

We found that the best accuracy results for the selected dataset were 63%-64% with similar performance for the models, especially in the LoRA condition. Essentially, we have shown that Gemma-2b can perform on par with RoBERTa for a given task, native for encoders.

Nevertheless, given RoBERTa performance was fairly similar for fine-tuned and for the LoRA condition, in the first case exceeding Gemma, and in the second case slightly behind it, we give the preference to it. RoBERTa has 20 times less parameters than Gemma-2b, which means that both training and inference will require less computational cost. Moreover, our best-performing Gemma model, as well as the best-performing RoBERTa were fine-tuned for this task explicitly, meaning they have lost their generalised purpose. Given that, there is no advantage to choosing the more computationally expensive fine-tuning of Gemma over the relatively quick fine-tuning of RoBERTa, at least until a more successful prompting technique is employed.

6.2 Limitations and Future Work

The systems tailored to classifying the political bias might suffer greatly from the lack of reliability or scalability. In particular, our system limitation is the spatially limited source of the data – the political environment of the USA. In the context of USA's politics, the distinction "left" or "right" might take the form of e.g. "pro-democrat" or "pro-republican", which loses scalability for other political contexts, outside of this environment. Future work should hence try to include datasets with a broader context, although then, the immediate difficulty arises of where to put the reference "centre" point. For example, say for some context outside of USA every public news coverage might seem as "right".

This highlights much broader disadvantage of training such systems. The perception of political bias depends on the Overton window, which is the range of politically accepted policies to the mainstream population in a given time [52]. This is the objective factor, and what an "objective" expert would label as "center" is located in the middle of this window. Not to discuss the impossibility of finding "objective expert", another issue is in place. The overton window shifts over time, and, for example, what once was "left" might become "right" to the mainstream. This highlights the temporal aspect of introducing systems as such. Perhaps, for future research a better alternative would be to train systems that detect the order of disagreement between two political texts, e.g. "How much does text A oppose/agree to text B?" This task, as opposed to ours, initially lacks an absolute starting point of predefined categories and handles the analysis purely on the basis of the relation of given texts.

References

- [1] Francisco-Javier Rodrigo-Ginés, Jorge Carrillo-de Albornoz, and Laura Plaza. A systematic review on media bias detection: What is media bias, how it is expressed, and how to detect it. *Expert Systems with Applications*, 237:121641, 2024.
- [2] Timo Spinde, Christin Jeggle, Magdalena Haupt, Wolfgang Gaissmaier, and Helge Giese. How do we raise media bias awareness effectively? effects of visualizations to communicate bias. *PLOS ONE*, 17(4):1–14, 04 2022.
- [3] News Center and Luke Auburn. Study of headlines shows media bias is growing, Mar 2024.
- [4] Daron R Shaw. The impact of news media favorability and candidate events in presidential campaigns. *Political Communication*, 16(2):183–202, 1999.
- [5] Jay J Van Bavel, Steve Rathje, Elizabeth Harris, Claire Robertson, and Anni Sternisko. How social media shapes polarization. *Trends in Cognitive Sciences*, 25(11):913–916, 2021.
- [6] Lian Parsons. Study finds political bias skews perceptions of verifiable fact, Nov 2023.
- [7] Smitha Milli, Micah Carroll, Yike Wang, Sashrika Pandey, Sebastian Zhao, and Anca D Dragan. Engagement, user satisfaction, and the amplification of divisive content on social media. *arXiv preprint arXiv:2305.16941*, 2023.
- [8] Nella Tiainen. Mitigating confirmation bias caused by social media algorithms. 2023.
- [9] Thomas J. Leeper and Rune Slothuus. 151How the News Media Persuades: Framing Effects and Beyond. In *The Oxford Handbook of Electoral Persuasion*. Oxford University Press, 06 2020.
- [10] Timo Spinde. An interdisciplinary approach for the automated detection and visualization of media bias in news articles. In *2021 International Conference on Data Mining Workshops (ICDMW)*, pages 1096–1103, 2021.
- [11] Shangbin Feng, Chan Young Park, Yuhan Liu, and Yulia Tsvetkov. From pretraining data to language models to downstream tasks: Tracking the trails of political biases leading to unfair NLP models. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki, editors, *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 11737–11762, Toronto, Canada, July 2023. Association for Computational Linguistics.
- [12] Marwan Omar, Soohyeon Choi, DaeHun Nyang, and David Mohaisen. Robust natural language processing: Recent advances, challenges, and future directions. *IEEE Access*, 10:86038–86056, 2022.
- [13] David Dukić and Jan Šnajder. Looking right is sometimes right: Investigating the capabilities of decoder-only llms for sequence labeling, 2024.
- [14] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large language models are zero-shot reasoners, 2023.
- [15] Monisha Kanakaraj and Ram Mohana Reddy Guddeti. Nlp based sentiment analysis on twitter data using ensemble classifiers. In *2015 3Rd international conference on signal processing, communication and networking (ICSCN)*, pages 1–5. IEEE, 2015.
- [16] Michael Crawford, Taghi M Khoshgoftaar, Joseph D Prusa, Aaron N Richter, and Hamzah Al Najada. Survey of review spam detection using machine learning techniques. *Journal of Big Data*, 2:1–24, 2015.
- [17] Trupti G Ghumade and RA Deshmukh. A document classification using nlp and recurrent neural network. *Int. J. Eng. Adv. Technol*, 8(6):632–636, 2019.

- [18] Kamilya Smagulova and Alex Pappachen James. A survey on lstm memristive neural network architectures and applications. *The European Physical Journal Special Topics*, 228(10):2313–2324, 2019.
- [19] Yong Yu, Xiaosheng Si, Changhua Hu, and Jianxun Zhang. A review of recurrent neural networks: Lstm cells and network architectures. *Neural computation*, 31(7):1235–1270, 2019.
- [20] Xuemei Bai. Text classification based on lstm and attention. In *2018 Thirteenth International Conference on Digital Information Management (ICDIM)*, pages 29–32. IEEE, 2018.
- [21] R Pascanu. On the difficulty of training recurrent neural networks. *arXiv preprint arXiv:1211.5063*, 2013.
- [22] Martin Jenckel, Syed Saqib Bukhari, and Andreas Dengel. Training lstm-rnn with imperfect transcription: limitations and outcomes. In *Proceedings of the 4th International Workshop on Historical Document Imaging and Processing*, pages 48–53, 2017.
- [23] Gokhan Soyalp, Artun Alar, Kaan Ozkanli, and Beytullah Yildiz. Improving text classification with transformer. In *2021 6th International Conference on Computer Science and Engineering (UBMK)*, pages 707–712. IEEE, 2021.
- [24] Ramy Baly, Giovanni Da San Martino, James Glass, and Preslav Nakov. We can detect your bias: Predicting the political ideology of news articles. *arXiv preprint arXiv:2010.05338*, 2020.
- [25] Frederick Liu, Siamak Shakeri, Hongkun Yu, and Jing Li. Enct5: Fine-tuning t5 encoder for non-autoregressive tasks. *arXiv preprint arXiv:2110.08426*, 2, 2021.
- [26] Abir Rahali and Moulay A Akhloufi. End-to-end transformer-based models in textual-based nlp. *AI*, 4(1):54–110, 2023.
- [27] Jesse Roberts. How powerful are decoder-only transformer neural models? In *2024 International Joint Conference on Neural Networks (IJCNN)*, pages 1–8. IEEE, 2024.
- [28] Chandrashekhar S Pawar and Ashwin Makwana. Comparison of bert-base and gpt-3 for marathi text classification. In *Futuristic Trends in Networks and Computing Technologies: Select Proceedings of Fourth International Conference on FTNCT 2021*, pages 563–574. Springer, 2022.
- [29] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach, 2019.
- [30] Yinhan Liu. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*, 364, 2019.
- [31] Gagan Reddy Narayanaswamy. Exploiting bert and roberta to improve performance for aspect based sentiment analysis. 2021.
- [32] Leila Behboudi Angizeh and Mohammad Reza Keyvanpour. Detecting fake news using advanced language models: Bert and roberta. In *2024 10th International Conference on Web Research (ICWR)*, pages 46–52, 2024.
- [33] Matthew E. Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee, and Luke Zettlemoyer. Deep contextualized word representations. *arXiv preprint arXiv:1802.05365*, 2018.
- [34] Gemma Team, Morgane Riviere, Shreya Pathak, Pier Giuseppe Sessa, Cassidy Hardin, Surya Bhupatiraju, Léonard Hussenot, Thomas Mesnard, Bobak Shahriari, Alexandre Ramé, et al. Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118*, 2024.

- [35] Jianlong Chen, Wei Xu, Zhicheng Ding, Jinxin Xu, Hao Yan, and Xinyu Zhang. Advancing prompt recovery in nlp: A deep dive into the integration of gemma-2b-it and phi2 models, 2024.
- [36] Hugging Face <https://huggingface.co/google/gemma-2-2b-it>.
- [37] Taku Kudo and John Richardson. Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In Eduardo Blanco and Wei Lu, editors, *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 66–71, Brussels, Belgium, Nov 2018. Association for Computational Linguistics.
- [38] Gemma Team, Thomas Mesnard, Cassidy Hardin, Robert Dadashi, Surya Bhupatiraju, Shreya Pathak, Laurent Sifre, Morgane Rivière, Mihir Sanjay Kale, Juliette Love, et al. Gemma: Open models based on gemini research and technology. *arXiv preprint arXiv:2403.08295*, 2024.
- [39] Nicolo Micheletti, Samuel Belkadi, Lifeng Han, and Goran Nenadic. Exploration of masked and causal language modelling for text generation, 2024.
- [40] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023.
- [41] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding, 2023.
- [42] Noam Shazeer. Glu variants improve transformer, 2020.
- [43] Biao Zhang and Rico Sennrich. Root mean square layer normalization, 2019.
- [44] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021.
- [45] Armen Aghajanyan, Luke Zettlemoyer, and Sonal Gupta. Intrinsic dimensionality explains the effectiveness of language model fine-tuning, 2020.
- [46] Iain J. Cruickshank and Lynnette Hui Xian Ng. Prompting and Fine-Tuning Open-Sourced Large Language Models for Stance Classification. *arXiv e-prints*, page arXiv:2309.13734, September 2023.
- [47] Xiaofei Sun, Xiaoya Li, Jiwei Li, Fei Wu, Shangwei Guo, Tianwei Zhang, and Guoyin Wang. Text classification via large language models. *arXiv preprint arXiv:2305.08377*, 2023.
- [48] AllSides. <https://www.allsides.com/headline-roundups>. Accessed: 2024-09-18.
- [49] Daniel Borkan, Lucas Dixon, Jeffrey Sorensen, Nithum Thain, and Lucy Vasserman. Nuanced metrics for measuring unintended bias with real data for text classification. In *Companion Proceedings of The 2019 World Wide Web Conference, WWW '19*, page 491–500, New York, NY, USA, 2019. Association for Computing Machinery.
- [50] Hengshuai Yao, Dong-lai Zhu, Bei Jiang, and Peng Yu. Negative log likelihood ratio loss for deep neural network classification. In *Proceedings of the Future Technologies Conference (FTC) 2019: Volume 1*, pages 276–282. Springer, 2020.
- [51] Liyuan Liu, Haoming Jiang, Pengcheng He, Weizhu Chen, Xiaodong Liu, Jianfeng Gao, and Jiawei Han. On the variance of the adaptive learning rate and beyond. *arXiv preprint arXiv:1908.03265*, 2019.
- [52] How the Politically Unthinkable Can Become Mainstream. <https://www.nytimes.com/2019/02/26/us/politics/overtone-window-democrats.html>. Accessed: 2024-10-31.